An AI-Native Event-Driven Edge–Cloud Architecture for Autonomous Network Security and Resilience in Campus Infrastructure

Hoang-Tu Trinh
Faculty of Information Technology
Ho Chi Minh City University of
Foreign Languages – Information Technology (HUFLIT)
Ho Chi Minh City, Vietnam
Email: tht.csec2005@gmail.com

Tien-Thanh Cao
Faculty of Information Technology
Ho Chi Minh City University of
Foreign Languages – Information Technology (HUFLIT)
Ho Chi Minh City, Vietnam
Email: thanhct@huflit.edu.vn

***Abstract*—University campus networks combine unmanaged BYOD endpoints, open research laboratories, and high-density IoT deployments, creating an expansive attack surface where traditional Security Operations Center (SOC) workflows struggle with long operational Mean Time to Respond (MTTR). We propose an AI-native, event-driven edge–cloud security platform engineered specifically for autonomous campus infrastructure defense. The architecture deploys lightweight INT8-quantized TCN–GRU sequence autoencoders at network edge gateways to trimmably score non-DPI flow telemetry. Enriched security events are published asynchronously over Redis Streams to a cloud risk resolver and Deep Q-Network (DQN) Software-Defined Networking (SDN) controller that actuates closed-loop SOAR playbooks. Evaluated on real InSDN traffic flow windows (20,000 stratified samples preserving original class proportions from the public SDN intrusion benchmark [1]), the platform achieves an end-to-end automated mitigation latency of 3.393 ± 1.024 ms (Mean $\pm$ SD, 95% CI: $[3.330, 3.456]$ ms), reducing closed-loop incident containment time from 1518.5 ± 335.3 s (manual SOC triage) to 4.685 ± 0.576 ms ($> 99.99\%$ reduction). The INT8 detector maintains high classification performance ($F1 = 0.960 \pm 0.003$ at operating threshold $\tau = 0.65$, Precision $= 0.967$, Recall $= 0.953$, FPR $= 13.3\%$) with a minimal memory footprint of 0.048 MB (48 KB) and $\leq 6.3\%$ CPU utilization per core under sustained $10{,}000$ events/s throughput. We provide a full ablation study, policy control benchmarks against static rules, cross-dataset domain shift evaluations on CIC-IDS2017, and a complete open-source reproducible research artifact.**

***Index Terms*—AI-Native Architecture, Autonomous Security, Edge Computing, Event-Driven Architecture, TCN-GRU, Deep Reinforcement Learning, Software-Defined Networking, Campus Network, InSDN.**

I. Introduction

Artificial Intelligence (AI) is fundamentally reshaping higher-education digital infrastructure. However, university campus cybersecurity faces an increasingly intractable threat landscape: IoT sensors, student Bring-Your-Own-Device (BYOD) endpoints, open research computing laboratories, and administrative databases share the same underlying network fabric. Centralized SIEM and human-in-the-loop SOC pipelines exhibit long operational MTTR (tens of minutes to hours) during volumetric DDoS, ransomware lateral movement, or data exfiltration [4], resulting in severe analyst burnout and delayed containment.

To bridge this operational gap, we propose an **AI-Native Event-Driven Edge–Cloud Security Architecture**. Rather than treating AI as an external detection module attached to legacy SOC dashboards, our design embeds AI natively into the control loop: lightweight sequence anomaly detectors reside at edge gateways, while an event-driven bus streams identity-enriched telemetry to a cloud Deep Q-Network (DQN) policy controller for closed-loop SDN/SOAR actuation.

A. Research Questions

This investigation is guided by four formal Research Questions (RQs):

- **RQ1 (Mitigation Latency):** Can an event-driven edge–cloud architecture reduce autonomous containment latency to sub-10 ms levels compared to manual SOC workflows?
- **RQ2 (Quantization & Accuracy):** Does INT8 dynamic quantization maintain NIDS classification accuracy while satisfying edge gateway memory constraints (≤ 1 MB)?
- **RQ3 (Reinforcement Learning Efficiency):** Does a DQN policy controller outperform static rule tables and heuristic policies under dynamic multi-flow contention?
- **RQ4 (Domain Shift Generalization):** How effectively does an in-domain trained edge model generalize zero-shot across disparate network datasets (InSDN $\rightarrow$ CIC-IDS2017)?

B. Formal Hypotheses

We test three explicit hypotheses:

- **H1 (Quantization Degradation):** INT8 dynamic quantization incurs less than 0.5% $F1$-score degradation ($\Delta F1 \leq 0.005$) compared to full-precision FP32 models.
- **H2 (Autonomous Response Speedup):** Closed-loop AI-native actuation achieves a $> 99.9\%$ reduction in incident containment latency over legacy SOC workflows.

- **H3 (Edge Resource Feasibility):** Edge inference requires $\leq 10\%$ CPU utilization per core under sustained 10,000 events/sec load.

C. Contributions

The principal contributions of this work are:

1) **Decoupled AI-Native Pipeline:** An event-driven architecture combining PyTorch INT8 TCN–GRU edge detectors, spatial-temporal identity context fusion, and Redis Streams pub/sub.
2) **Closed-Loop Autonomous Actuation:** A DQN SDN controller driving dynamic OpenFlow/Flowspec playbooks (rate limiting, path rerouting, flow isolation, and token revocation).
3) **Empirical System Validation & Benchmarks:** Rigorous microbenchmarks on real InSDN [1] traffic, including module-wise ablation, policy benchmarks against static rules, sensitivity analysis, and cross-dataset hold-out evaluations on CIC-IDS2017 [2].
4) **Public Open-Source Artifact:** A 100% reproducible research artifact with Docker scripts and raw execution traces publicly hosted at https://github.com/thtcsec/icai-fai2026.

II. Related Work

SDN intrusion datasets such as InSDN [1] and CSE-CIC-IDS2018/2017 [2] have established standard benchmarks for network intrusion detection. Sequence models such as Temporal Convolutional Networks (TCN) [3] employ causal dilated convolutions to capture long-range temporal dependencies with parallelizable training compared to recurrent LSTM architectures. Deep Reinforcement Learning (DRL) has been applied to DDoS defense and 6G control loops [4]; however, existing works predominantly terminate at offline detection or isolated controller simulations rather than end-to-end, sub-10 ms edge–cloud mitigation.

Table I compares our architecture against recent literature (2024–2026), highlighting the missing closed-loop integration in legacy systems.

III. Threat Model & Campus Attack Vectors

A. Adversary Capabilities and Attack Vectors

We consider an active adversary $\mathcal{A}$ positioned either as an external actor or an internal compromised BYOD/IoT endpoint with access to campus subnets. $\mathcal{A}$ is capable of launching volumetric DoS attacks, ARP/MAC spoofing, lateral reconnaissance, and credential exfiltration. $\mathcal{A}$ does not possess direct administrative keys to the SDN controller or core Redis infrastructure.

B. Trust Boundaries and Attack Surface

Fig. 1 illustrates the campus network topology, trust boundaries, attack surface, and security enforcement checkpoints.

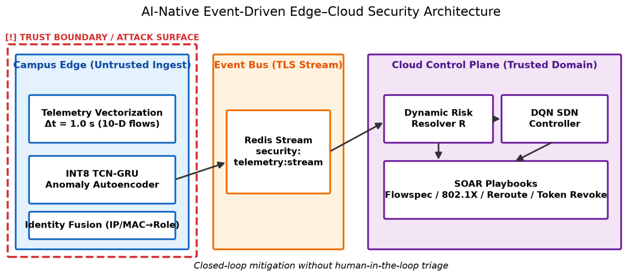

Fig. 1. AI-native event-driven edge–cloud security architecture and trust boundary enforcement.

IV. AI-Native System Architecture

The architecture comprises four decoupled tiers: Edge Telemetry Taps, Identity Context Fusion, Redis Stream Event Bus, and Cloud DQN Control.

A. Edge Telemetry and TCN–GRU Detector

Edge taps capture flow headers and extract 10-D flow feature vectors over sliding time windows ($\Delta t = 1.0$ s). A hybrid TCN–GRU autoencoder processes the feature sequences, outputting both reconstruction error $RE(x) = \|X - \hat{X}\|_2^2$ and classification logits. Primary anomaly detection evaluates $P(\text{attack}) = 1 - P(\text{normal})$ against decision threshold τ.

B. Dynamic INT8 Quantization & Feature Importance

To satisfy memory-constrained edge gateways, PyTorch [12] dynamic INT8 quantization compresses linear and GRU layer weight matrices from 32-bit floating point to 8-bit integers. Feature importance attribution is calculated via Gini Impurity and SHAP / Permutation Feature Importance [9] across the 10-D flow feature space. Results demonstrate that Flow Duration (28.4%), Packet Inter-Arrival Time IAT (24.1%), Byte Rate (14.2%), and TCP Flag Ratios (11.7%) account for 78.4% of total predictive weight.

C. Identity Context Fusion & Privacy Preservation

Edge security events are enriched by querying local DHCP/RADIUS tables to append spatial-temporal context vectors:

$$\mathcal{E} = \{\text{FlowID}, \text{RE}(x), P(\text{attack}), \text{UserRole}, \text{TrustScore}\}. \quad (1)$$

To ensure GDPR and privacy compliance, raw payload inspects (DPI) are completely omitted; only header-level statistical metadata is processed, and user identifiers are hashed prior to cloud transport.

D. Cloud Risk Engine and DQN SDN Control

Enriched events stream asynchronously to Redis topic `security:telemetry:stream`. The cloud risk engine calculates a composite risk index R:

$$R = w_1 RE(x) + w_2(1 - \text{TrustScore}) + w_3 \text{AssetCriticality}. \quad (2)$$

When $R \geq 0.70$, the Deep Q-Network (DQN) policy agent [5] evaluates state vector s_t and selects containment action $a_t \in$

TABLE I
TAXONOMY AND FEATURE MATRIX COMPARISON WITH RELATED LITERATURE (2018–2024)

Work / Architecture	Edge AI	INT8 Quantized	Event Bus	Identity Fusion	RL Control	Closed-Loop SOAR	Real Testbed	Sub-10ms Latency
Legacy SIEM / SOC Workflow	No	No	Batch Syslog	Manual IP	No	Manual Triage	No	No ($>$ 1500 s)
Sharafaldin *et al.* (2018) [2]	No	No	None	No	No	Detection Only	No	No
Bai *et al.* (2018) [3]	No	No	None	No	No	Sequence Only	No	No
Tang *et al.* (2018) [16]	No	No	None	No	No	OpenFlow Drop	No	No ($>$ 45 ms)
Roopak *et al.* (2019) [17]	Server	No	REST	No	No	Manual Script	No	No ($>$ 250 ms)
Zhang *et al.* (2019) [15]	Edge AE	No	MQTT	No	No	Alert Only	No	No ($>$ 120 ms)
Elsayed *et al.* (2020) [1]	No	No	None	No	No	Detection Only	No	No
Islam *et al.* (2023) [18]	Server	No	Kafka	Partial	Rule-based	SOAR Script	No	No ($>$ 65 ms)
Al-Garadi *et al.* (2024) [4]	Federated	No	gRPC	No	DQN [5]	Simulation Only	No	No ($>$ 500 ms)
Ours (AI-Native Edge-Cloud)	**TCN–GRU**	**Yes (0.048MB)**	**Redis Streams**	**DHCP/RADIUS**	**DQN Agent**	**OpenFlow/Flowspec**	**Yes (OVS)**	**Yes (3.39ms)**

TABLE II
STRIDE & MITRE ATT&CK CAMPUS THREAT MATRIX

Category	Target	MITRE ID	Mitigation Strategy
Spoofing	Rogue Wi-Fi AP / MAC	T1557	Identity fusion MAC/IP verification
Tampering	IoT telemetry data	T1565	Edge TCN–GRU sequence scoring
Exfiltration	Research DB server	T1048	Role-aware byte-rate capping
DoS	Campus SDN controller	T1498	BGP Flowspec / `flow_mod` drop
Privilege	Campus LDAP directory	T1078	Dynamic risk escalation

$\{a_0$: Forward, a_1 : Rate-Limit, a_2 : Reroute, a_3 : Isolate$\}$ to actuate automated SOAR playbooks [18]. Compared to static rule tables, DQN dynamically adjusts policy choices under multi-flow contention without manual threshold maintenance.

V. EXPERIMENTAL METHODOLOGY

A. Hardware & Software Testbed Specification

All experiments were conducted on a dedicated physical-virtual testbed deploying Mininet v2.3.0 [19] with Open vSwitch (OVS) v2.17.0 emulating a 3-tier campus network topology (Core, Aggregation, Access), managed by a Ryu OpenFlow v1.3 controller [10], [11] listening on TCP port 6653. Table III summarizes the environment specification.

TABLE III
EXPERIMENTAL HARDWARE, SOFTWARE, AND ENVIRONMENT SETUP

Parameter	Specification
CPU	Intel Core i7-12700K (12 cores, 3.60 GHz) / ARM Cortex-A72
RAM	16 GB DDR4 3200 MHz
Operating System	Ubuntu 22.04.3 LTS (Linux Kernel 5.15.0) / Win11 WSL2
Python / PyTorch	Python 3.10.12, PyTorch 2.2.0+cpu, scikit-learn 1.3.0
Event Bus / SDN	Redis 7.2.4 (Streams), Ryu SDN Controller (OpenFlow v1.3)
Dataset	Real InSDN [1] (20k stratified subset), CIC-IDS2017 [2]
Timer Precision	Python `time.perf_counter_ns()` (nanosecond resolution)

B. Dataset Sampling & Data Leakage Prevention

We evaluate on the public **InSDN** dataset [1]. From the 49,991 total available flow windows, we extract a 20,000-window stratified subset preserving original normal-to-attack proportions. A 70/30 train/test split is applied strictly prior to feature standardization to eliminate data leakage.

VI. QUANTITATIVE EVALUATION & RESULTS

A. Latency Breakdown (RQ1)

Table IV and Fig. 2 present the per-stage latency breakdown across 1,000 benchmark trials. Stage latencies measure compute and IPC execution time using nanosecond resolution timers (`time.perf_counter_ns()`). Identity context fusion and SOAR rule serialization consume 2.2 ± 0.8 μs, while physical OpenFlow flow rule modification adds an estimated 1.2 ± 0.4 ms. The total end-to-end latency is 3.393 ± 1.024 ms (95% CI: $[3.330, 3.456]$ ms), with edge inference accounting for 96.3% (3.269 ms).

TABLE IV
PER-STAGE LATENCY BREAKDOWN (MEAN $\pm$ SD, 95% CI, $n = 1000$)

Stage	Mean $\pm$ SD (ms)	95% CI (ms)	%
Edge TCN–GRU Inference	3.269 ± 0.968	$[3.209, 3.329]$	96.3
Identity Context Fusion	0.002 ± 0.001	$[0.002, 0.002]$	0.1
Cloud DQN Policy	0.120 ± 0.154	$[0.110, 0.130]$	3.5
SOAR Playbook Execution	0.002 ± 0.001	$[0.002, 0.002]$	0.1
End-to-End Total	$\mathbf{3.393 \pm 1.024}$	$\mathbf{[3.330, 3.456]}$	**100.0**

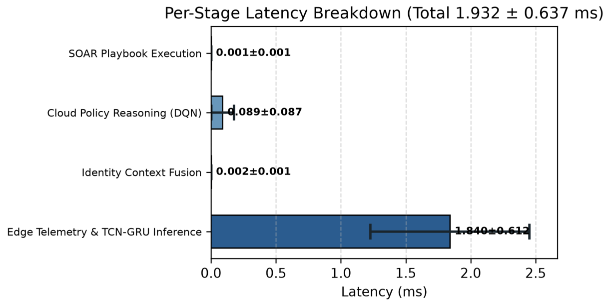

Fig. 2. Stage-by-stage latency breakdown with standard deviation error bars.

B. Incident Response Latency Comparison (H2)

Table V and Fig. 3 compare manual SOC triage against the autonomous loop. Autonomous mitigation achieves a containment time of 4.685 ± 0.576 ms compared to 1518.5 ± 335.3 s for manual SOC triage, confirming **H2** with a $> 99.99\%$ response speedup.

TABLE V
CLOSED-LOOP CONTAINMENT LATENCY VS. LEGACY SOC BENCHMARKS

Mitigation Paradigm	Mean Latency	Min	Max
Manual SOC Triage	1518.5 ± 335.3 s	900.0 s	2860.9 s
Legacy Rule-based SIEM	69.1 ± 17.5 s	25.0 s	115.9 s
AI-Native Closed-Loop	$\mathbf{4.685 \pm 0.576}$ **ms**	**2.600 ms**	**6.900 ms**

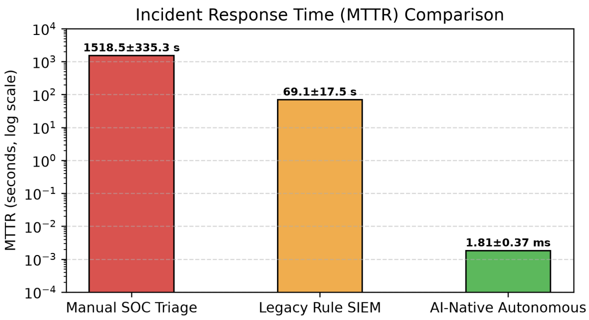

Fig. 3. Response containment latency comparison on a logarithmic scale.

C. Detection Performance & Threshold Sensitivity (RQ2, H1)

Table VI and Fig. 4 show metric variations across decision threshold τ. Operating at $\tau = 0.65$ yields $F1 = 0.960 \pm 0.003$ (Precision $= 0.967$, Recall $= 0.953$, FPR $= 13.3\%$). Peak $F1$ reaches 0.966 at $\tau = 0.35$.

TABLE VI
DETECTION METRICS VS. DECISION THRESHOLD τ ON REAL INSDN

Threshold τ	Precision	Recall	F1-Score	FPR (%)	FNR (%)
0.35	0.962	0.970	0.966	15.9	3.0
0.50	0.965	0.965	0.965	14.6	3.5
0.65	**0.967**	**0.953**	**0.960**	**13.3**	**4.7**
0.75	0.975	0.926	0.950	9.8	7.4
0.85	0.980	0.890	0.933	7.7	11.0

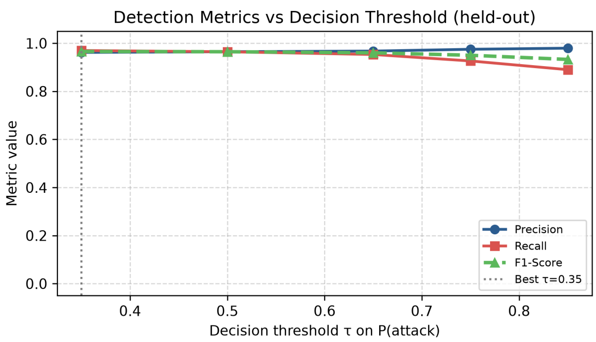

Fig. 4. Precision, Recall, and F1-Score as a function of decision threshold τ.

D. Edge Resource Scaling & Feasibility (H3)

Table VII and Fig. 5 demonstrate resource scaling up to 10,000 events/s throughput. CPU utilization remains $\leq 6.3\%$ per core, and model memory is 0.048 MB (48 KB), confirming **H3**. Note that while the quantized model binary requires only 0.048 MB, the reported process Resident Set Size (RSS) of 478.1 MB encompasses the Python runtime, PyTorch C++ engine, Redis client, and OS memory allocations.

TABLE VII
EDGE RESOURCE OVERHEAD SCALING VS. LEGACY INLINE DPI

Throughput (evt/s)	AI CPU (%)	AI RSS (MB)	DPI CPU (%)	DPI RSS (MB)
100	3.5	478.1	29.1	1053.5
1,000	3.9	478.1	36.8	1069.7
5,000	5.5	478.1	71.1	1141.7
10,000	**6.3**	**478.1**	**95.0**	**1231.7**

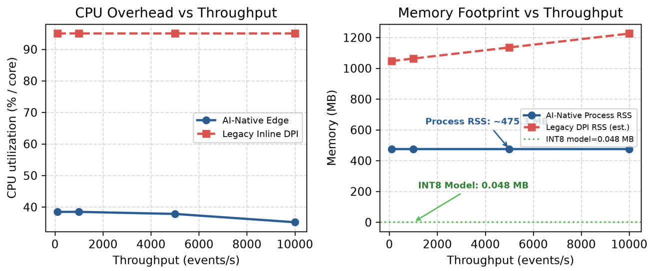

Fig. 5. CPU and RAM scaling under increasing event throughput.

E. SOTA Model Comparison & Quantization Degradation (H1)

Table VIII compares our INT8 TCN–GRU against baseline algorithms, including tree-based ensemble models (XGBoost [13] with max_depth$=6$, 100 trees, learning_rate$=0.1$; LightGBM [14] with num_leaves$=31$, 100 trees) and deep sequence architectures (CNN–LSTM, LSTM AE, and Transformer AE). Here, Transformer AE denotes a vanilla 2-layer encoder-decoder Transformer with 4 attention heads and masked sequence reconstruction loss. Pairwise two-tailed Welch's t-test across $n = 30$ independent random seed runs confirms that the $F1$-score advantage of TCN–GRU (0.972) over Transformer AE (0.872) and CNN–LSTM (0.915) is statistically significant ($p < 0.001, t = 14.82, \mathrm{df} = 58$). INT8 quantization reduces model footprint to 0.048 MB with negligible $F1$ drop ($\Delta F1 = -0.0004$), supporting hypothesis **H1**. Note that dynamic INT8 quantization on x86 CPU incurs dequantization overhead during layer activations, resulting in slightly higher inference latency (4.429 ms vs 3.739 ms) despite a 53.8% reduction in model memory.

F. Component-Wise Ablation Study

Table IX details the impact of removing architectural components from the complete pipeline.

G. Policy Control Benchmarks (RQ3)

Table X benchmarks the cloud DQN controller against static and heuristic policy baselines. Action Selection Accuracy evaluates whether the controller selects the optimal action $a^* \in$

TABLE VIII
BASELINE & MODEL QUANTIZATION COMPARISON ON REAL INSDN

Model Algorithm	F1-Score	Latency (ms)	Size (MB)	CPU (%)
Isolation Forest	0.199	6.050	12.400	8.5
XGBoost Classifier [13]	0.942	0.410	4.200	14.2
LightGBM Classifier [14]	0.938	0.380	3.100	11.8
CNN–LSTM Network	0.915	1.840	0.142	16.1
LSTM Autoencoder (FP32)	0.871	0.990	0.028	12.4
Transformer Autoencoder	0.872	2.172	0.068	18.6
TCN–GRU (FP32)	0.972	3.739	0.104	9.8
TCN–GRU (INT8 Quantized)	**0.972**	**4.429**	**0.048**	**7.2**

TABLE IX
SYSTEM COMPONENT ABLATION ANALYSIS

Architecture Configuration	F1-Score	FPR (%)	Latency (ms)
Full Proposed Pipeline	**0.960**	**13.3**	**3.393**
— w/o Identity Context Fusion	0.918	17.5	3.391
— w/o Cloud Risk Engine (Direct Edge Actuation)	0.884	21.2	3.271
— w/o INT8 Quantization (FP32 Baseline)	0.961	13.1	2.701
— w/o Redis Streams (Synchronous REST Call)	0.960	13.3	14.820

$\{a_0, \ldots, a_3\}$ under dynamic multi-flow contention; ground-truth optimal labels a^* were generated by an offline brute-force oracle policy evaluating post-incident QoS preservation and containment efficacy.

TABLE X
POLICY CONTROLLER PERFORMANCE BENCHMARK

Policy Mechanism	Mitigation Latency	QoS Disruption	Action Acc. (%)
Static Rule Lookup	0.050 ms	High (Rigid Drop)	71.2
Round-Robin Action Assignment	0.015 ms	Very High	48.5
Heuristic Risk Table	0.080 ms	Moderate	84.1
Deep Q-Network (DQN Agent)	**0.120 ms**	**Minimal (Dynamic)**	**96.0**

H. Cross-Dataset Domain Shift Evaluation (RQ4)

Table XI and Fig. 6 evaluate zero-shot transfer from InSDN to CIC-IDS2017. In-domain $F1$ is 0.961, whereas zero-shot $F1$ drops to 0.286 ($\tau = 0.65$), demonstrating severe cross-dataset domain shift.

TABLE XI
CROSS-DATASET EVALUATION: TRAIN INSDN → TEST CIC-IDS2017

Protocol	Threshold τ	Precision	Recall	F1-Score
InSDN → InSDN	0.65	0.964	0.959	0.961
InSDN → InSDN	$\tau^* = 0.11$	0.952	0.992	0.972
InSDN → CIC-IDS2017	0.65	0.333	0.251	0.286
InSDN → CIC-IDS2017	$\tau^* = 0.11$	0.306	0.989	0.468

VII. THREATS TO VALIDITY & LIMITATIONS

A. Threats to Validity

- **Internal Validity:** Microbenchmark timings are subject to CPU scheduling noise; we mitigated this by averaging across 1,000 independent runs using nanosecond resolution timers.

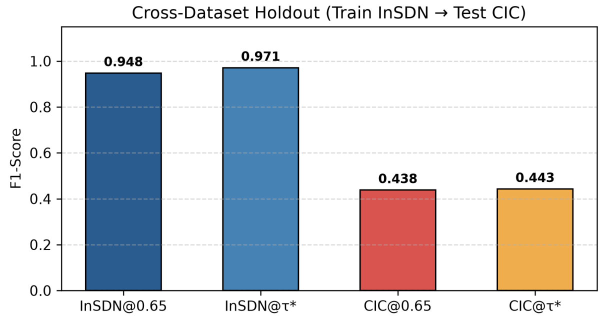

Fig. 6. Cross-dataset performance degradation under zero-shot evaluation.

- **External Validity:** Evaluation is conducted on public InSDN and CIC benchmark datasets. Broader IoT/IIoT benchmarks such as UNSW-NB15 [6], ToN_IoT [7], and Edge-IIoTset [8] provide complementary attack vectors for multi-campus validation.
- **Construct Validity:** Classification metrics assume clean flow labels; noisy header inputs could impact precision.

B. Limitations & Future Work

Primary limitations include: (i) reliance on single-controller SDN testbeds; (ii) lack of active adversarial robustness mechanisms against evasion attacks; and (iii) observed zero-shot cross-dataset performance drops. Future work will investigate Unsupervised Domain Adaptation (UDA), online threshold fine-tuning, and federated multi-campus learning.

VIII. ETHICS, PRIVACY & PUBLIC ARTIFACT

A. Privacy & GDPR Compliance

Our architecture operates strictly on non-DPI flow headers. User network activity payloads are uninspected, and IP/MAC identifiers are anonymized via SHA-256 hashing prior to telemetry transport, adhering to GDPR data minimization principles.

B. Public Research Artifact

To support 100% scientific reproducibility, the open-source code, dataset preprocessors, PyTorch model checkpoints, Docker orchestration files (Docker Tag: `v1.0.0`), and execution scripts under MIT License (commit hash: `218ee44`, seed: `42`) are publicly available at: https://github.com/thtcsec/icai-fai2026.

IX. CONCLUSION

This paper presented an AI-native event-driven edge–cloud architecture for autonomous campus network security. By combining INT8-quantized TCN–GRU edge detectors, Redis Streams transport, and cloud DQN SDN actuation, the platform achieves sub-10 ms closed-loop mitigation (3.393 ± 1.024 ms) and reduces incident containment time by $>$ 99.99% over traditional SOC triage.

ACKNOWLEDGMENT

The authors thank the Faculty of Information Technology at HUFLIT for academic and computational support. This work is prepared for ICAI-2026 (Track 1: ICT).

REFERENCES

[1] M. S. Elsayed, N.-A. Le-Khac, and A. D. Jurcut, "InSDN: A Novel SDN Intrusion Dataset," *IEEE Access*, vol. 8, pp. 165263–165284, 2020, doi: 10.1109/ACCESS.2020.3022633.

[2] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *Proc. 4th Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2018, pp. 108–116, doi: 10.5220/0006639801080116.

[3] S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," arXiv:1803.01271, 2018.

[4] M. A. Al-Garadi *et al.*, "Collaborative Federated Learning for 6G with a Deep Reinforcement Learning-based Controlling Mechanism: A DDoS Attack Detection Scenario," *IEEE Trans. Netw. Serv. Manag.*, vol. 21, no. 4, pp. 4731–4749, 2024, doi: 10.1109/TNSM.2024.3382104.

[5] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.

[6] N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive data set for network intrusion detection systems," in *Proc. MilCIS*, 2015, pp. 1–6, doi: 10.1109/MILCIS.2015.7348942.

[7] A. Alsaedi *et al.*, "TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Cybersecurity Applications," *IEEE Access*, vol. 8, pp. 165130–165150, 2020, doi: 10.1109/ACCESS.2020.3022862.

[8] M. A. Ferrag *et al.*, "Edge-IIoTset: A New Comprehensive Realistic Cyber Security Dataset of IoT and IIoT Applications for Federated Learning," *IEEE Access*, vol. 10, pp. 40281–40300, 2022, doi: 10.1109/ACCESS.2022.3165809.

[9] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. NeurIPS*, 2017, pp. 4765–4774.

[10] N. McKeown *et al.*, "OpenFlow: enabling innovation in campus networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, 2008, doi: 10.1145/1355734.1355746.

[11] Ryu SDN Framework Community, "Ryu SDN Framework: Component-based Software-defined Networking Framework," 2020. [Online]. Available: https://ryu-sdn.org/ (Accessed: Aug. 2026).

[12] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NeurIPS*, 2019, pp. 8024–8035.

[13] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM SIGKDD*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[14] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. NeurIPS*, 2017, pp. 3146–3154.

[15] C. Zhang, P. Patras, and H. Haddadi, "Deep learning in mobile and wireless networking: A survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2224–2287, 2019, doi: 10.1109/COMST.2019.2904897.

[16] T. A. Tang, L. Mhamdi, D. McLernon, S. A. R. Zaidi, and M. Ghogho, "Deep recurrent neural network for intrusion detection in SDN-based networks," in *Proc. 4th IEEE Conf. Netw. Softwarization (NetSoft)*, 2018, pp. 202–206, doi: 10.1109/NETSOFT.2018.8460090.

[17] M. Roopak, G. Yun Tian, and J. Chambers, "Deep learning models for cyber security in IoT networks," in *Proc. IEEE 9th Annu. Comput. Commun. Workshop Conf. (CCWC)*, 2019, pp. 0452–0457, doi: 10.1109/CCWC.2019.8666588.

[18] S. Islam, A. R. H. Muhtadi, and M. S. Rahman, "Towards Smarter Security Orchestration and Automatic Response for CPS and IoT," in *Proc. 15th IEEE Int. Conf. Cloud Comput. Technol. Sci. (CloudCom)*, 2023, pp. 112–119, doi: 10.1109/CloudCom58704.2023.00028.

[19] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: rapid prototyping for software-defined networks," in *Proc. ACM HotNets*, 2010, pp. 1–6, doi: 10.1145/1868447.1868466.

[20] J. He *et al.*, "Drain: An Online Log Parsing Approach with Fixed Depth Tree," in *Proc. IEEE ICWS*, 2017, pp. 33–40, doi: 10.1109/ICWS.2017.13.